Universidad Católica de Temuco

INFORME DE LABORATORIO 2

Expresiones Regulares con JFLAP

Teoría de la Computación

Dato	**Información**
Código curso	INFO1148
Integrantes	Nombre: ALEXIS MONSALVE Nombre: CHRISTOHPER SOLIS
Profesor	Prof. M. Lévano
Fecha de entrega	11/10/2026

Índice

1. Introduccion **2**
- 1.1. Resultados de Aprendizaje . . . 2
- 1.2. Indicaciones Generales . . . 2

2. Pregunta 1: Interpretación y validación de una expresión regular **3**
- 2.1. Interpretación del lenguaje . . . 3
- 2.2. Conversión de la expresión regular a AFN . . . 3
- 2.3. Predicción y validación de cadenas . . . 4
- 2.4. Justificación de resultados . . . 5
  - 2.4.1. Cadena 1101 . . . 5
  - 2.4.2. Cadena 0100 . . . 5
- 2.5. Conclusión de la pregunta 1 . . . 6

3. Pregunta 2: Diseño de una expresión regular **6**
- 3.1. Diseño de la expresión regular . . . 6
- 3.2. Justificación de la expresión regular . . . 6
- 3.3. Conversión de la expresión regular a AFN . . . 7
- 3.4. Pruebas y validación de cadenas . . . 8
- 3.5. Análisis de casos límite . . . 9
- 3.6. Conclusión de la pregunta 2 . . . 9

4. Pregunta 3: Conversión ER → AFN → AFD **9**
- 4.1. Diseño de la expresión regular . . . 9
- 4.2. Conversión de la expresión regular a AFN . . . 10
- 4.3. Conversión de AFN a AFD . . . 10
- 4.4. Comparación de características . . . 10
- 4.5. Ejecución de cadenas en AFN y AFD . . . 11
- 4.6. Análisis de equivalencia . . . 12
- 4.7. Conclusión de la pregunta 3 . . . 12

5. Pregunta 4: Problema de diseño y validación **13**
- 5.1. Diseño de la expresión regular . . . 13
- 5.2. Estrategia utilizada . . . 13
- 5.3. Implementación en JFLAP . . . 13
- 5.4. Conversión de la expresión regular a AFN . . . 14
- 5.5. Conversión de AFN a AFD . . . 14
- 5.6. Pruebas y validación de cadenas . . . 15
- 5.7. Análisis de casos límite . . . 16
- 5.8. Capacidad de reconocimiento de ER, AFN y AFD . . . 16
- 5.9. Conclusión de la pregunta 4 . . . 17

6. Anexos **17**
- 6.1. Enlace a los archivos . . . 17

1. Introduccion

Aplicar los conceptos de lenguajes regulares y expresiones regulares mediante JFLAP, relacionando una expresión regular con los autómatas finitos que reconocen el mismo lenguaje. El laboratorio busca que el estudiante formule soluciones, realice conversiones y valide experimentalmente sus resultados.

1.1. Resultados de Aprendizaje

- Interpretar el lenguaje descrito por una expresión regular.
- Diseñar expresiones regulares a partir de restricciones sobre cadenas.
- Convertir expresiones regulares a autómatas finitos no deterministas (AFN) utilizando JFLAP.
- Convertir un AFN a un autómata finito determinista (AFD).
- Comprobar, mediante casos de prueba, la equivalencia entre distintas representaciones de un lenguaje regular.
- Justificar formalmente los resultados observados en JFLAP.

1.2. Indicaciones Generales

- El trabajo se debe desarrollar utilizando JFLAP.
- Antes de ejecutar una prueba en JFLAP, el estudiante deberá indicar el resultado esperado cuando la pregunta lo solicite.
- No se evaluarán únicamente capturas de pantalla: cada resultado debe acompañarse de una breve explicación.
- Las expresiones regulares deben ser diseñadas por los estudiantes y escritas utilizando la sintaxis compatible con JFLAP.
- Para la unión, JFLAP utiliza el símbolo +; para la cerradura de Kleene, el símbolo *.
- Los archivos .jff deben abrir correctamente y corresponder con los autómatas presentados en el informe.
- Las capturas deben ser legibles y mostrar claramente los estados, transiciones y estados finales.
- En las preguntas que solicitan casos de prueba propios, deben incluirse tanto cadenas aceptadas como rechazadas.

2. Pregunta 1: Interpretación y validación de una expresión regular

2.1. Interpretación del lenguaje

Se considera el alfabeto binario:

$$\Sigma = \{0, 1\}$$

Y la expresión regular:

$$R_1 = (0 + 1)^*01$$

La expresión regular representa el lenguaje formado por todas las cadenas binarias que terminan en la secuencia `01`.

La expresión puede interpretarse mediante dos componentes:

- $(0 + 1)^*$**:** permite cualquier combinación de ceros y unos, incluyendo la cadena vacía.
- 01**:** establece que los dos últimos símbolos de la cadena deben ser un cero seguido de un uno.

Por lo tanto, el lenguaje reconocido se define como:

$$L_1 = \{w \in \{0, 1\}^* \mid w \text{ termina en } 01\}$$

La cadena de menor longitud que pertenece al lenguaje es `01`.

2.2. Conversión de la expresión regular a AFN

Para realizar la conversión se utiliza la herramienta JFLAP, ingresando la siguiente expresión regular:

$$R_1 = (0 + 1)^*01$$

La conversión permite obtener un Autómata Finito No Determinista (AFN) que reconoce el lenguaje de cadenas binarias terminadas en 01.

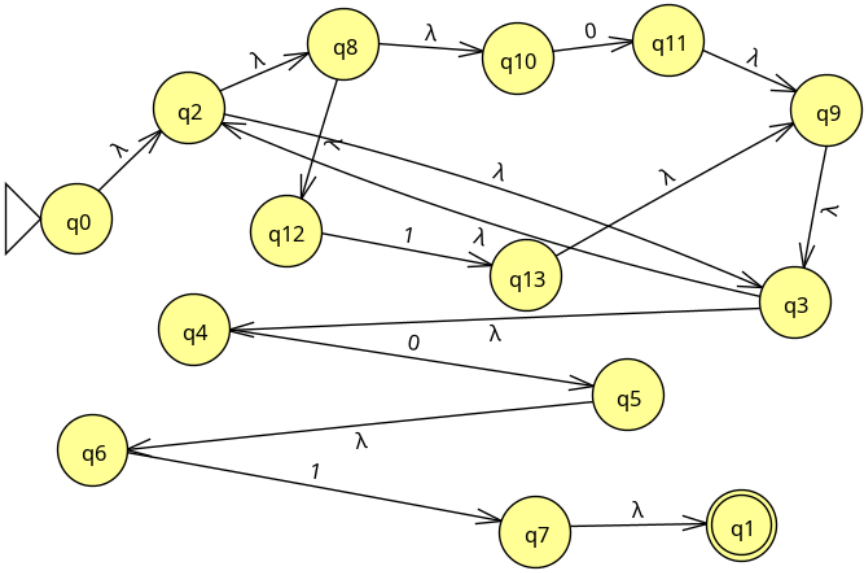


Figura 1: AFN generado mediante JFLAP.

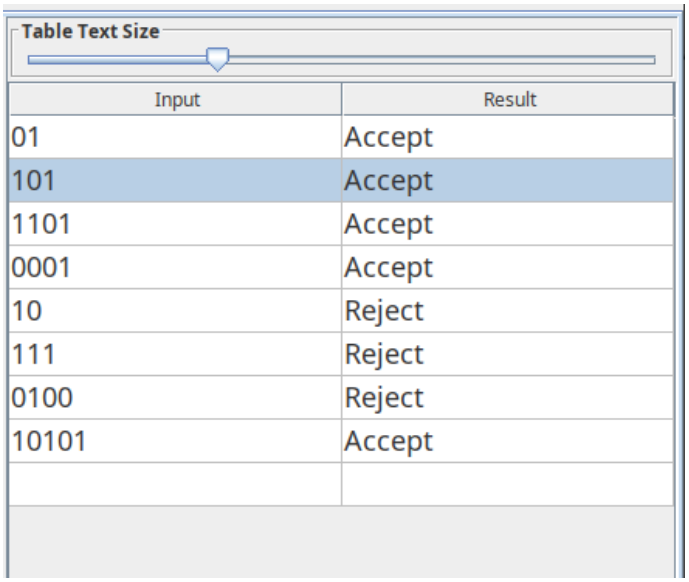
2.3. Predicción y validación de cadenas

Para comprobar el funcionamiento del autómata, se analizaron ocho cadenas pertenecientes al alfabeto $\Sigma = \{0, 1\}$.

Antes de realizar las pruebas en JFLAP, se establecieron las predicciones considerando que toda cadena aceptada debe terminar en la secuencia `01`.

Cadena	**Predicción**	**Resultado JFLAP**	**¿Coinciden?**
01	Aceptada	Aceptada	Sí
101	Aceptada	Aceptada	Sí
1101	Aceptada	Aceptada	Sí
0001	Aceptada	Aceptada	Sí
10	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
0100	Rechazada	Rechazada	Sí
10101	Aceptada	Aceptada	Sí

Cuadro 1: Comparación entre las predicciones teóricas y los resultados obtenidos en JFLAP.



Input	Result
01	Accept
101	Accept
1101	Accept
0001	Accept
10	Reject
111	Reject
0100	Reject
10101	Accept

Figura 2: Resultados de las ocho cadenas evaluadas mediante Multiple Run en JFLAP.

2.4. Justificación de resultados

2.4.1. Cadena 1101

La cadena `1101` es aceptada porque puede descomponerse de la siguiente forma:

$$1101 = \underbrace{11}_{(0+1)^*} \cdot \underbrace{01}_{\text{terminación}}$$

El prefijo `11` cumple con la cerradura de Kleene, puesto que pertenece al conjunto de combinaciones posibles de ceros y unos.

Además, los últimos dos símbolos de la cadena son `01`.

Por lo tanto:

$$1101 \in L_1$$

Esta predicción fue confirmada mediante la ejecución del AFN en JFLAP.

2.4.2. Cadena 0100

La cadena `0100` es rechazada debido a que sus dos últimos símbolos son `00`.

Aunque la cadena está formada por símbolos pertenecientes al alfabeto, no cumple la condición de terminación.

En consecuencia:

$$0100 \notin L_1$$

La ejecución del autómata en JFLAP confirmó el rechazo de esta cadena.

2.5. Conclusión de la pregunta 1

Mediante el análisis de la expresión regular $(0+1)^*01$, se determinó que el lenguaje corresponde al conjunto de cadenas binarias que terminan en la secuencia `01`.

La conversión realizada en JFLAP permitió obtener un Autómata Finito No Determinista (AFN) que reconoce dicho lenguaje.

Durante la validación experimental se evaluaron ocho cadenas diferentes, obteniéndose cinco cadenas aceptadas y tres rechazadas.

Los resultados experimentales coincidieron con las predicciones teóricas en los ocho casos evaluados, alcanzando un 100 % de coincidencia.

Por lo tanto, las pruebas realizadas respaldan el funcionamiento del autómata para las cadenas consideradas y su correspondencia con la expresión regular.

3. Pregunta 2: Diseño de una expresión regular

3.1. Diseño de la expresión regular

Se considera el lenguaje:

$$L_2 = \{w \in \{a,b\}^* \mid w \text{ comienza con } a \text{ y termina con } b\}$$

El lenguaje establece dos condiciones: las cadenas deben comenzar con el símbolo `a` y terminar con el símbolo `b`.

Para representar este lenguaje se propone la siguiente expresión regular:

$$\boxed{R_2 = a(a+b)^*b}$$

3.2. Justificación de la expresión regular

La expresión está compuesta por tres elementos fundamentales:

- **Símbolo inicial** a**:** garantiza que toda cadena reconocida comience con la letra `a`.
- **Secuencia intermedia** $(a+b)^*$**:** permite cualquier combinación de los símbolos `a` y `b`, incluyendo la cadena vacía.
- **Símbolo final** b**:** obliga a que todas las cadenas reconocidas terminen con la letra `b`.

Por lo tanto, la expresión propuesta satisface simultáneamente las dos restricciones establecidas.

Además, como la secuencia intermedia puede ser vacía, la cadena de menor longitud aceptada es:

$$w_{\text{mín}} = ab$$

3.3. Conversión de la expresión regular a AFN

La expresión regular propuesta fue ingresada en la herramienta JFLAP:

$$R_2 = a(a+b)^*b$$

Posteriormente, se realizó la conversión a un Autómata Finito No Determinista (AFN).

El autómata obtenido contiene 14 estados, desde q_0 hasta q_{13}.

El estado inicial corresponde a q_0, mientras que q_1 representa el estado de aceptación.

Se identifican transiciones con los símbolos a y b, además de transiciones λ que permiten cambiar de estado sin consumir caracteres de entrada.

La estructura del autómata permite reconocer cadenas que comienzan con a y terminan con b, admitiendo cualquier combinación intermedia de símbolos del alfabeto.

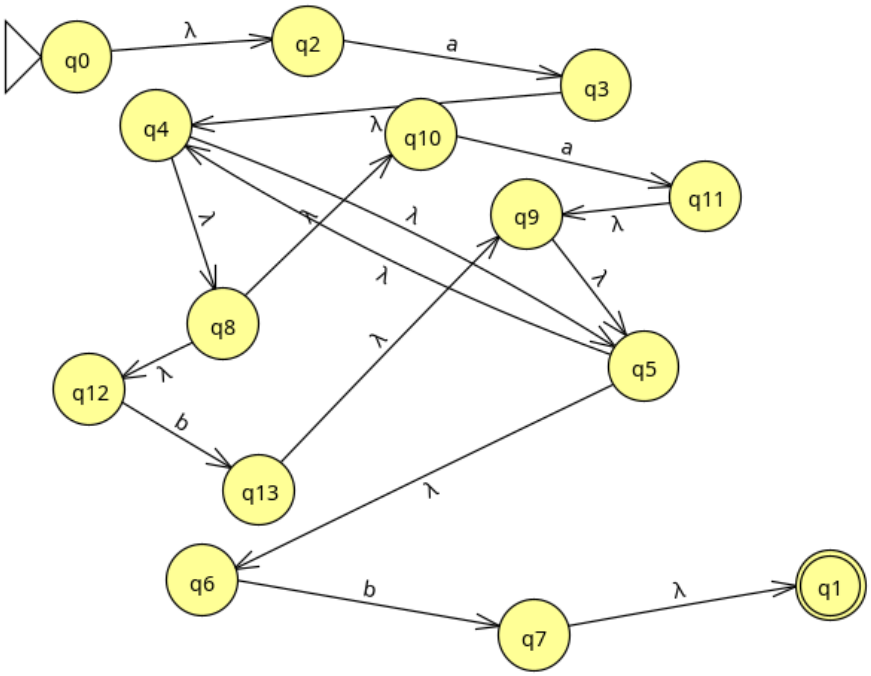


Figura 3: AFN generado mediante JFLAP para la expresión regular $a(a+b)^*b$.

3.4. Pruebas y validación de cadenas

Para validar el comportamiento del autómata, se seleccionaron doce cadenas de prueba, incluyendo cadenas aceptadas, rechazadas, la cadena de longitud mínima y casos límite.

Antes de ejecutar las pruebas en JFLAP, se predijo el resultado esperado de cada cadena de acuerdo con las condiciones del lenguaje.

Cadena	**Resultado esperado**	**Resultado JFLAP**
`ab`	Aceptada	Aceptada
`aab`	Aceptada	Aceptada
`abb`	Aceptada	Aceptada
`abab`	Aceptada	Aceptada
`aaabb`	Aceptada	Aceptada
`a`	Rechazada	Rechazada
`b`	Rechazada	Rechazada
`aa`	Rechazada	Rechazada
`bb`	Rechazada	Rechazada
`baab`	Rechazada	Rechazada
`aba`	Rechazada	Rechazada
ε	Rechazada	Rechazada

Cuadro 2: Comparación entre los resultados esperados y los resultados obtenidos en JFLAP.

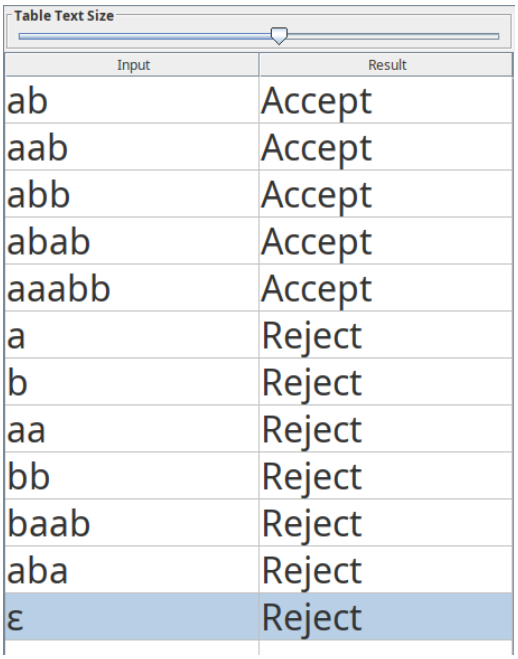


Figura 4: Resultados de las pruebas ejecutadas en JFLAP mediante Multiple Run.

3.5. Análisis de casos límite

Entre las cadenas evaluadas se consideran los siguientes casos límite:

- **Cadena mínima** `ab`**:** es aceptada porque comienza con `a` y termina con `b`. Además, es la cadena de menor longitud que satisface ambas condiciones.
- **Cadena** `a`**:** es rechazada porque, aunque comienza correctamente con `a`, no termina con `b`.
- **Cadena** `b`**:** es rechazada porque termina con `b`, pero no comienza con `a`.
- **Cadena vacía** ε**:** es rechazada porque no posee símbolo inicial ni símbolo final, por lo que no puede satisfacer ninguna de las restricciones del lenguaje.

3.6. Conclusión de la pregunta 2

A partir de las condiciones del lenguaje, se diseñó la expresión regular $a(a+b)^*b$, la cual describe correctamente el conjunto de cadenas que comienzan con `a` y terminan con `b`.

La conversión de esta expresión en JFLAP permitió obtener un AFN equivalente, el cual fue validado mediante doce cadenas de prueba.

Los resultados obtenidos en JFLAP coincidieron en todos los casos con las predicciones teóricas, lo que confirma que la expresión regular propuesta representa correctamente el lenguaje solicitado.

En consecuencia, existe consistencia entre la definición formal del lenguaje, la expresión regular diseñada y el AFN generado.

4. Pregunta 3: Conversión ER $\rightarrow$ AFN $\rightarrow$ AFD

4.1. Diseño de la expresión regular

Se considera el lenguaje:

$$L_3 = \{\, w \in \{0,1\}^* \mid w \text{ contiene la subcadena } 101 \,\}$$

Para representar este lenguaje se propone la siguiente expresión regular:

$$R_3 = (0+1)^*101(0+1)^*$$

La expresión está compuesta por tres elementos:

- **Prefijo** $(0+1)^*$**:** permite cualquier combinación de ceros y unos antes de la subcadena, incluyendo la cadena vacía.
- **Subcadena** 101**:** obliga a que los símbolos 1, 0 y 1 aparezcan de forma consecutiva.

- **Sufijo** $(0+1)^*$**:** permite cualquier combinación de ceros y unos después de la subcadena, incluyendo la cadena vacía.

De esta forma, toda cadena generada por R_3 contiene la subcadena 101 y, recíprocamente, toda cadena que contiene 101 puede descomponerse como prefijo · 101 · sufijo. La cadena de menor longitud que pertenece al lenguaje es:

$$w_{\text{mín}} = 101$$

4.2. Conversión de la expresión regular a AFN

El AFN obtenido contiene 24 estados, desde q_0 hasta q_{23}. El estado inicial corresponde a q_0, mientras que q_1 es el único estado de aceptación. JFLAP construye el autómata combinando pequeños autómatas para cada símbolo, unión y cerradura de Kleene, enlazados mediante transiciones λ. En el diagrama se distinguen tres partes: el bloque del prefijo $(0+1)^*$, la secuencia lineal que lee los símbolos 1, 0 y 1, y el bloque del sufijo $(0+1)^*$ que conduce al estado final q_1.

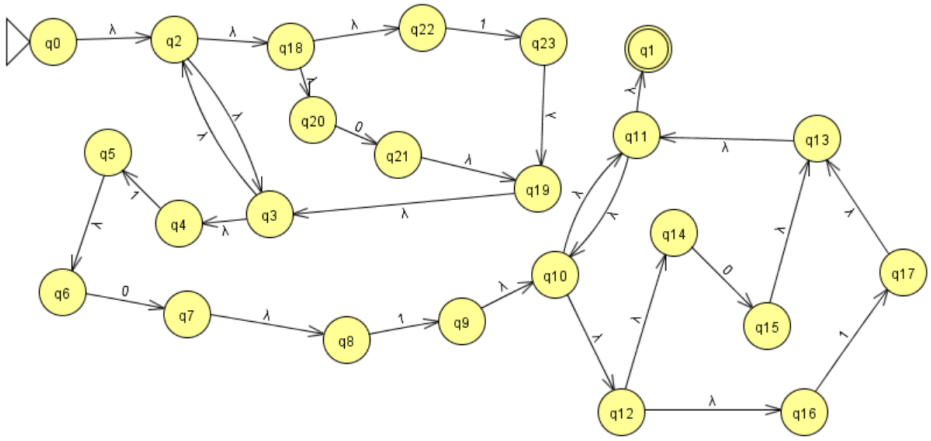


Figura 5: AFN generado mediante JFLAP para la expresión regular $(0+1)^*101(0+1)^*$.

4.3. Conversión de AFN a AFD

JFLAP realiza esta conversión mediante la construcción de subconjuntos: cada estado del AFD representa un conjunto de estados del AFN, incluida su cerradura λ, lo que se observa en la etiqueta de cada estado. Por ejemplo, el estado inicial del AFD corresponde al conjunto $\{0, 2, 14, 3, 16, 18, 4\}$ de estados del AFN. Un estado del AFD es de aceptación cuando su conjunto contiene el estado final q_1 del AFN.

El AFD obtenido contiene 9 estados, desde q_0 hasta q_8, de los cuales q_4, q_5, q_6, q_7 y q_8 son de aceptación.

4.4. Comparación de características

El AFN no es determinista porque posee transiciones λ y, desde algunos estados, puede seguir más de un camino con la misma entrada. El AFD, en cambio, tiene

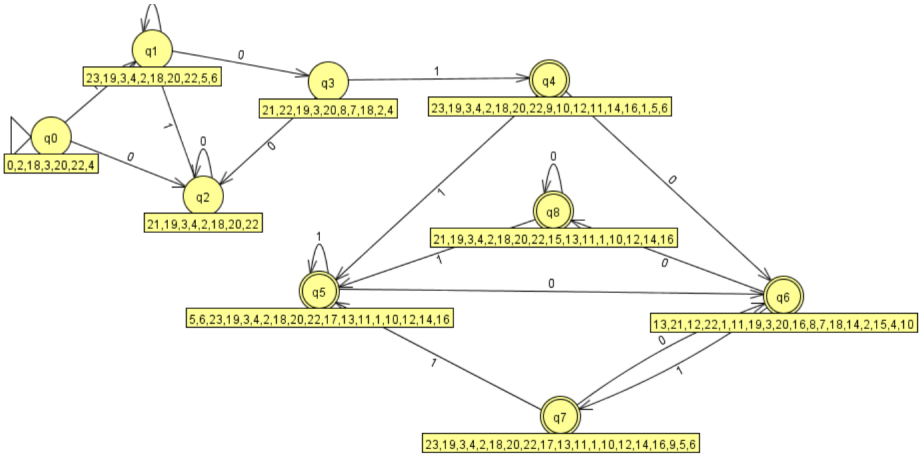


Figura 6: AFD obtenido en JFLAP a partir del AFN de la pregunta 3.

Característica	**AFN**	**AFD**
N.º de estados	24	9
N.º de transiciones	29 (22 λ y 7 con símbolo)	18
Transiciones λ	Sí	No
Estados de aceptación	1 (q_1)	5 (q_4 a q_8)
¿Es determinista?	No	Sí

Cuadro 3: Comparación de las características del AFN y del AFD de la pregunta 3.

exactamente una transición con 0 y una con 1 desde cada uno de sus 9 estados ($9 \times 2 = 18$ transiciones) y no posee transiciones λ.

Cabe destacar que el AFD tiene más estados que el mínimo posible para este lenguaje (4 estados), ya que JFLAP no minimiza el autómata al realizar la conversión. Esto no afecta el lenguaje reconocido.

4.5. Ejecución de cadenas en AFN y AFD

Las ocho cadenas indicadas se ejecutaron en ambos autómatas mediante la opción *Input* → *Multiple Run* de JFLAP.

Input	Result
101	Accept
0101	Accept
1010	Accept
11011	Accept
000	Reject
111	Reject
1001	Reject
1101010	Accept

Input	Result
101	Accept
0101	Accept
1010	Accept
11011	Accept
000	Reject
111	Reject
1001	Reject
1101010	Accept

Figura 7: Resultados de Multiple Run en JFLAP: AFN (izquierda) y AFD (derecha).

Cadena	AFN	AFD	¿Mismo resultado?
101	Aceptada	Aceptada	Sí
0101	Aceptada	Aceptada	Sí
1010	Aceptada	Aceptada	Sí
11011	Aceptada	Aceptada	Sí
000	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
1001	Rechazada	Rechazada	Sí
1101010	Aceptada	Aceptada	Sí

Cuadro 4: Resultados de las cadenas ejecutadas en el AFN y en el AFD de la pregunta 3.

Las cadenas aceptadas contienen la subcadena 101 en alguna posición. Por ejemplo:

$$11011 = \underbrace{1}_{(0+1)^*} \cdot \underbrace{101}_{\text{subcadena}} \cdot \underbrace{1}_{(0+1)^*}$$

En cambio, las cadenas 000 y 111 se rechazan porque están formadas por un único símbolo repetido, por lo que no pueden contener la secuencia 101. La cadena 1001 también se rechaza, ya que los dos ceros consecutivos separan a los unos e impiden formar la subcadena 101.

4.6. Análisis de equivalencia

El AFN y el AFD **no** reconocen lenguajes diferentes: ambos reconocen exactamente el lenguaje L_3.

La construcción de subconjuntos garantiza que, después de leer una cadena, el AFD queda en el estado que representa todos los estados en los que podría encontrarse el AFN tras leer esa misma cadena. El AFD acepta cuando dicho conjunto contiene algún estado final del AFN, es decir, cuando existe al menos un camino de aceptación en el AFN. Por lo tanto, ambos autómatas aceptan y rechazan exactamente las mismas cadenas.

La conversión solo modifica la forma del autómata: elimina las transiciones λ y el no determinismo, a cambio de un número distinto de estados, pero no altera el lenguaje reconocido. Esto fue confirmado experimentalmente, ya que las ocho cadenas evaluadas obtuvieron el mismo resultado en ambos autómatas.

4.7. Conclusión de la pregunta 3

Se diseñó la expresión regular $(0+1)^*101(0+1)^*$ para el lenguaje de cadenas binarias que contienen la subcadena 101. Mediante JFLAP se realizaron las conversiones ER $\rightarrow$ AFN, obteniendo un autómata de 24 estados con transiciones λ, y AFN $\rightarrow$ AFD, obteniendo un autómata determinista de 9 estados sin transiciones λ.

Las ocho cadenas evaluadas obtuvieron el mismo resultado en ambos autómatas, alcanzando un 100 % de coincidencia. En consecuencia, el AFN y el AFD son equivalentes y reconocen el mismo lenguaje L_3.

5. Pregunta 4: Problema de diseño y validación

5.1. Diseño de la expresión regular

Se considera el alfabeto $\Sigma = \{a, b\}$ y el lenguaje formado por las cadenas que cumplen simultáneamente dos condiciones:

$$L_4 = \{\, w \in \{a,b\}^* \mid w \text{ contiene al menos una } a \text{ y termina en } bb \,\}$$

Para representar este lenguaje se propone la siguiente expresión regular:

$$R_4 = b^*a(a+b)^*bb$$

5.2. Estrategia utilizada

La estrategia consiste en dividir la cadena a partir de su primera a. La expresión está compuesta por cuatro elementos:

- b^***:** representa todas las b que pueden aparecer antes de la primera a, incluyendo ninguna.
- a**:** corresponde a la primera a de la cadena y garantiza que exista al menos una.
- $(a+b)^*$**:** permite cualquier combinación de símbolos después de esa primera a.
- bb**:** obliga a que la cadena termine en bb.

Como los elementos se concatenan, toda cadena generada por R_4 cumple ambas condiciones de forma simultánea. Además, la expresión no deja fuera ninguna cadena válida: si una cadena termina en bb, sus dos últimos símbolos son b, por lo que la a que contiene necesariamente se encuentra antes de ellos. Así, toda cadena válida puede escribirse como $b^* \cdot a \cdot (a+b)^* \cdot bb$. La cadena de menor longitud que pertenece al lenguaje es:

$$w_{\text{mín}} = abb$$

5.3. Implementación en JFLAP

La expresión regular fue ingresada en JFLAP mediante la opción *Regular Expression*, utilizando la sintaxis compatible con la herramienta: el símbolo `+` para la unión y el símbolo `*` para la cerradura de Kleene. La expresión ingresada fue `b*a(a+b)*bb`

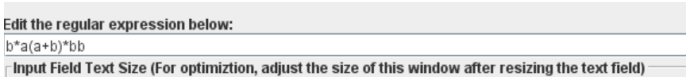


Figura 8: Expresión regular $b^*a(a+b)^*bb$ ingresada en JFLAP.

5.4. Conversión de la expresión regular a AFN

El AFN obtenido contiene 20 estados, desde q_0 hasta q_{19}. El estado inicial corresponde a q_0 y el único estado de aceptación es q_1. En el diagrama se identifican el bloque de la cerradura b^*, la transición que lee la a obligatoria, el bloque de la cerradura $(a+b)^*$ y, finalmente, las dos transiciones con b que conducen al estado q_1.

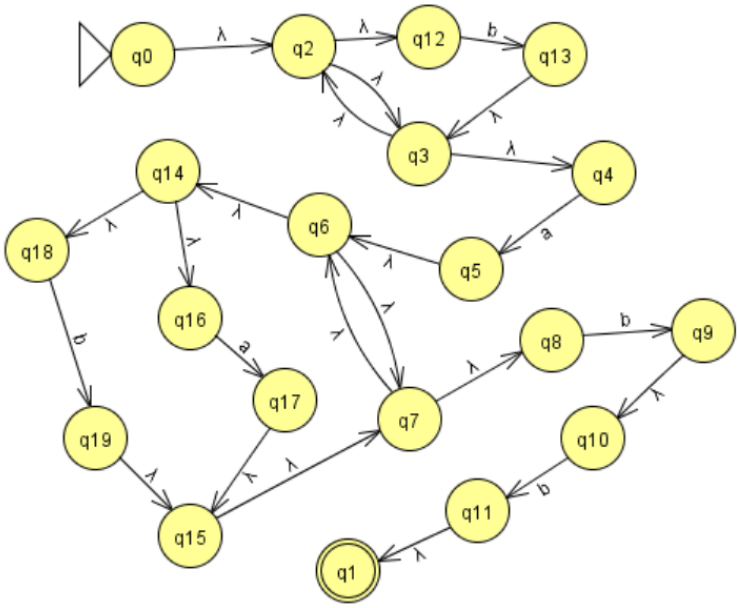


Figura 9: AFN generado mediante JFLAP para la expresión regular $b^*a(a+b)^*bb$.

5.5. Conversión de AFN a AFD

El AFD obtenido contiene 6 estados, desde q_0 hasta q_5, donde q_5 es el único estado de aceptación. Su estado inicial corresponde al conjunto $\{0, 2, 12, 3, 4\}$ de estados del AFN. El comportamiento de sus estados puede interpretarse de la siguiente forma:

- q_0 **y** q_1**:** aún no se ha leído ninguna a.
- q_2 **y** q_4**:** ya se leyó al menos una a y el último símbolo leído es a.
- q_3**:** ya se leyó al menos una a y la cadena termina en una sola b.
- q_5**:** ya se leyó al menos una a y la cadena termina en bb (estado de aceptación).

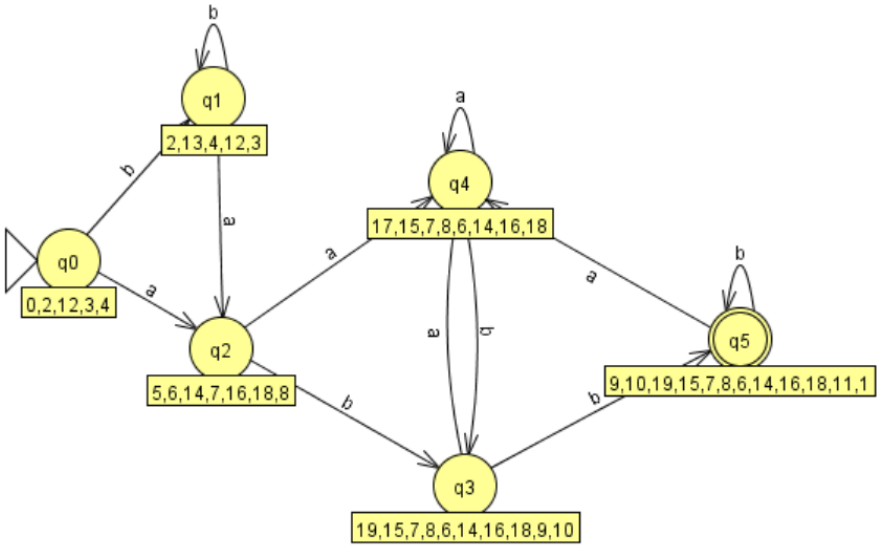


Figura 10: AFD obtenido en JFLAP a partir del AFN de la pregunta 4.

Característica	**AFN**	**AFD**
N.º de estados	20	6
N.º de transiciones	24 (18 λ y 6 con símbolo)	12
Transiciones λ	Sí	No
Estados de aceptación	1 (q_1)	1 (q_5)
¿Es determinista?	No	Sí

Cuadro 5: Comparación de las características del AFN y del AFD de la pregunta 4.

5.6. Pruebas y validación de cadenas

Para validar el lenguaje se seleccionaron doce cadenas: cinco aceptadas, cinco rechazadas y dos casos límite. Antes de ejecutar las pruebas en JFLAP se predijo el resultado esperado de cada cadena según las condiciones del lenguaje. Luego, cada cadena se ejecutó tanto en el AFN como en el AFD mediante *Multiple Run*.

Input	Result
abb	Accept
aabb	Accept
babbb	Accept
ababbb	Accept
bbbabb	Accept
bb	Reject
aaa	Reject
aba	Reject
baba	Reject
abab	Reject
	Reject
ab	Reject

Input	Result
abb	Accept
aabb	Accept
babbb	Accept
ababbb	Accept
bbbabb	Accept
bb	Reject
aaa	Reject
aba	Reject
baba	Reject
abab	Reject
	Reject
ab	Reject

Figura 11: Resultados de Multiple Run en JFLAP: AFN (izquierda) y AFD (derecha). La fila vacía corresponde a la cadena vacía λ.

Las cadenas aceptadas contienen al menos una a y terminan en bb. Por ejemplo:

$$babbb = \underbrace{b}_{b^*} \cdot \underbrace{a}_{\text{primera } a} \cdot \underbrace{b}_{(a+b)^*} \cdot \underbrace{bb}_{\text{terminación}}$$

Cadena	Resultado esperado	AFN	AFD
`abb`	Aceptada	Aceptada	Aceptada
`aabb`	Aceptada	Aceptada	Aceptada
`babbb`	Aceptada	Aceptada	Aceptada
`ababbb`	Aceptada	Aceptada	Aceptada
`bbbabb`	Aceptada	Aceptada	Aceptada
`bb`	Rechazada	Rechazada	Rechazada
`aaa`	Rechazada	Rechazada	Rechazada
`aba`	Rechazada	Rechazada	Rechazada
`baba`	Rechazada	Rechazada	Rechazada
`abab`	Rechazada	Rechazada	Rechazada
λ	Rechazada	Rechazada	Rechazada
`ab`	Rechazada	Rechazada	Rechazada

Cuadro 6: Comparación entre el resultado esperado y los resultados en el AFN y el AFD de la pregunta 4.

Las cadenas rechazadas incumplen al menos una de las condiciones: `bb` termina en *bb* pero no contiene ninguna *a*, mientras que `aaa`, `aba`, `baba` y `abab` contienen *a* pero no terminan en *bb*.

5.7. Análisis de casos límite

Se definieron los siguientes casos límite:

- **Cadena vacía λ:** es rechazada porque tiene longitud cero, por lo que no contiene ninguna *a* ni termina en *bb*.
- **Cadena `ab`:** es rechazada porque, aunque contiene una *a*, solo termina en una *b*. Se encuentra a un símbolo de la cadena mínima aceptada, por lo que comprueba que el autómata exige las dos *b* finales.

Además, la cadena `abb` corresponde a la cadena de menor longitud aceptada, y fue reconocida correctamente por ambos autómatas.

5.8. Capacidad de reconocimiento de ER, AFN y AFD

Expresión regular, AFN y AFD no poseen distinta capacidad de reconocimiento: son formas diferentes de representar la misma clase de lenguajes, los lenguajes regulares. El teorema de Kleene establece que todo lenguaje descrito por una expresión regular puede ser reconocido por un AFN, que todo AFN puede transformarse en un AFD equivalente mediante la construcción de subconjuntos y que, a su vez, todo AFD puede expresarse como una expresión regular. En este laboratorio se comprobó en la práctica: JFLAP transformó la expresión regular en un AFN con transiciones λ y luego en un AFD sin λ ni no determinismo, y las doce cadenas probadas obtuvieron el mismo resultado en ambos autómatas. Las diferencias están en la forma: la expresión regular es compacta, el AFN es fácil de construir y el AFD es directo de ejecutar.

5.9. Conclusión de la pregunta 4

Se diseñó la expresión regular $b^*a(a+b)^*bb$, que garantiza simultáneamente la presencia de al menos una a y la terminación en bb. Mediante JFLAP se realizaron las conversiones ER $\rightarrow$ AFN, obteniendo un autómata de 20 estados, y AFN $\rightarrow$ AFD, obteniendo un autómata determinista de 6 estados.

Las doce cadenas evaluadas, incluidos los casos límite, coincidieron con el resultado esperado tanto en el AFN como en el AFD. En consecuencia, existe consistencia entre la definición del lenguaje, la expresión regular diseñada y los autómatas generados.

6. Anexos

6.1. Enlace a los archivos

Los archivos `.jff` se encuentran disponibles en el siguiente enlace:

https://github.com/alexhozt/lab2